

Introduction

The feature-store benchmark suite exercises DuckDB's CREATE FEATURE / REFRESH FEATURE / SERVE FEATURE surface against the [ClickBench hits dataset](#), reshaped as a clickstream feature store keyed by UserID. The suite treats every benchmark as a point in a multi-dimensional "knob space" (e.g. source scale, aggregation complexity, retention depth, TTL, serve mode) and samples that space with a seeded pairwise covering array; this is the same combinatorial-interaction-testing technique found in randomised DB testing such as SQLancer. The emitted scenario set contains every pairwise combination of knob values that can co-occur at least once, while avoiding a combinatorial explosion. This gives broad, reproducible coverage of the feature-store's behavior space in a tractable number of runs. The tests are generated by a fixed --seed (42), and three benchmark families are generated using [scripts/generate_feature_benchmarks.py](#):

1. [serve/](#) — times SERVE FEATURE/SERVE FEATURES (point-in-time and ASOF time-travel retrieval), pairwise-sampled across 10 dimensions. 21 scenarios.
2. [refresh/](#) — times a single REFRESH FEATURE ... AT <ts> (the windowed group-by snapshot build), pairwise-sampled across 5 dimensions. 16 scenarios.
3. [cases/](#) — 6 curated, hand-designed multi-step scenarios (multi-version refresh history, GC/eviction pressure, refresh-then-serve, concurrent multi-feature refresh, interleaved refresh+serve, and steady-state high-retain eviction) run at each of 3 source scales. 18 scenarios total.

Dataset reference

The benchmark source comes from the [ClickBench hits dataset](#), and the PARQUET_COUNT knob controls how many ClickBench hits_N.parquet files are loaded as the source table, giving three fixed scale points used throughout all three families:

Parquet files	Total number of rows	Distinct Users
1	1,000,000	79,842
10	10,000,000	1,530,334
100	99,997,497	17,630,976

Table 1: Metrics for the size of the source tables benchmarked in ClickBench hits table.

Dimensions

REFRESH

Dimension	Values	What it controls
PARQUET_COUNT	1, 10, 100	Source scale ($\approx$ 1M/10M/100M rows) — main driver of raw scan cost
AGG	simple, multi, distinct, window	Aggregation shape: plain count / multi-stat / count-distinct / window function — stresses different executor paths
WINDOW	1 DAY, 30 DAYS, 3650 DAYS	Width of the time-window filter — controls how much of the source table one refresh must touch
RETAIN	1, 5, 20	Versions kept before eviction — the GC-volume knob
PRIOR_VERSIONS	0, 5	Refreshes already built (untimed) before the timed one — how much existing history the timed refresh contends with

Table 2: Dimension knobs for generation of REFRESH benchmarks.

SERVE

Dimension	Values	What it controls
PARQUET_COUNT	1, 10, 100	Source scale ($\approx$ 1M/10M/100M rows)
AGG	simple, multi, distinct, window	Aggregation shape of the feature being served
RETAIN	1, 5, 20	Versions kept before eviction
N_REFRESH	1, 5, 20	Refreshes built (untimed) before the timed serve — version depth being searched
TTL	0, 30 DAYS	Staleness cutoff off/on — toggles the NULL-projection path for stale features
SPINE_SIZE	10k, 100k	Number of entities requested per serve call (read batch size)
SERVE_MODE	latest, asof	Latest-value probe vs. time-travel ASOF lookup
SERVE_TS_POS	na, before, mid, latest	Where the spine's request timestamp sits vs. the snapshot history (only meaningful under asof; forced na under latest)
N_FEATURES	1, 3	Single SERVE FEATURE vs. SERVE FEATURES a,b,c
WRAP	bare, where, groupby, cte	How the SERVE result is composed in SQL — probes optimizer pushdown behavior

Table 3: Dimension knobs for generation of SERVE benchmarks.

CASES

Scenario	What it tests
multi_version_refresh	8 refreshes in one timed unit, RETAIN 20 — no eviction fires; pure version-growth cost
gc_pressure_refresh	Same 8 refreshes, RETAIN 2 — every refresh past v2 evicts; GC always firing, store stays small
high_retain_gc_pressure	Store pre-built to 10 versions (RETAIN 10), then 8 more timed refreshes — every one evicts from an already-large, steady-state store
refresh_then_serve	One refresh immediately followed by a point-in-time ASOF serve — write-then-read latency
concurrent_feature_refresh	Three features sharing the same entity key, refreshed back to back in one timed unit
refresh_serve_interleaved	Three repeated refresh+serve cycles back to back — steady-state interleaved read/write

Table 4: Description of each handwritten case scenario, each scenario is run at all three source scales (1/10/100 parquet files).

Environment

All benchmark results were collected on a single machine (Intel Core Ultra 9 185H, 11 cores / 22 threads, 15 GiB RAM) running Ubuntu 22.04.5 LTS under WSL2. Absolute timings should be read as directional (useful for regression comparison run-to-run on the *same* machine) rather than as portable, comparable-across-machines numbers.

Results

Each value is the mean of 3 runs. Times in seconds, rounded off to 3 decimal places. New time represents benchmark ran with optimizations, refer to the poster / report for more details.

SERVE

PARQUET_COUNT	AGG	RETAIN	N_REFRESH	TTL	SPINE_SIZE	SERVE_MODE	N_FEATURES	WRAP	OLD TIME (s)	NEW TIME (s)	% SPEEDUP
1	distinct	20	1	30 DAYS	100k	latest-na	3	bare	0.023	0.032	-39.6%
1	multi	20	20	30 DAYS	10k	asof-latest	3	groupby	0.094	0.018	+80.4%
1	simple	1	1	30 DAYS	100k	asof-latest	3	cte	0.020	0.017	+16.0%
1	simple	20	5	0	100k	asof-before	3	where	0.015	0.016	-4.8%
1	window	5	1	0	100k	asof-mid	1	cte	0.022	0.016	+27.8%
10	distinct	1	5	30 DAYS	10k	asof-mid	3	where	0.014	0.015	-6.9%
10	distinct	5	1	0	10k	asof-before	1	groupby	0.013	0.012	+9.4%
10	distinct	20	5	0	10k	asof-latest	3	cte	0.425	0.034	+92.0%
10	multi	1	5	0	100k	asof-latest	1	groupby	0.043	0.021	+51.6%

10	multi	5	1	30 DAYS	10k	latest-na	1	cte	0.041	0.011	+72.0%
10	multi	20	1	0	100k	asof-before	3	where	0.015	0.018	-15.5%
10	simple	1	20	0	10k	latest-na	1	bare	0.045	0.015	+67.4%
10	window	20	20	30 DAYS	100k	asof-mid	3	groupby	1.541	0.064	+95.8%
100	distinct	5	20	0	100k	asof-latest	1	where	1.620	0.047	+97.1%
100	multi	5	5	30 DAYS	100k	asof-mid	1	bare	1.962	0.056	+97.1%
100	simple	5	20	30 DAYS	10k	asof-before	3	cte	0.023	0.154	-557.0%
100	simple	20	1	0	10k	latest-na	1	groupby	0.295	0.039	+86.8%
100	simple	20	20	0	10k	asof-mid	3	cte	133.505	0.265	+99.8%
100	window	1	1	0	100k	asof-before	3	bare	0.027	0.071	-159.2%
100	window	20	5	0	10k	latest-na	1	where	2.231	0.186	+91.7%
100	window	20	20	0	100k	asof-latest	1	bare	26.261	0.057	+99.8%

Table 5: Benchmark results for Serve test cases, with specified dimension configurations.

REFRESH

PARQUET_COUNT	AGG	WINDOW	RETAIN	PRIOR_VERSIONS	OLD TIME (s)	NEW TIME (s)	% SPEEDUP
1	distinct	30 DAYS	5	5	0.031	0.035	-11.2%
1	distinct	3650 DAYS	1	5	0.030	0.034	-10.4%
1	multi	1 DAY	5	0	0.023	0.022	+3.6%
1	simple	3650 DAYS	20	0	0.022	0.023	-4.6%
1	window	1 DAY	5	5	0.023	0.021	+10.7%
10	distinct	1 DAY	20	0	0.167	0.107	+35.6%
10	multi	30 DAYS	20	5	0.309	0.219	+28.9%
10	multi	3650 DAYS	1	0	0.346	0.263	+23.8%
10	simple	30 DAYS	20	5	0.177	0.129	+27.4%
10	window	3650 DAYS	5	5	0.363	0.348	+4.0%
100	distinct	30 DAYS	20	5	1.697	1.525	+10.1%
100	multi	1 DAY	1	5	0.970	0.846	+12.8%
100	simple	1 DAY	1	0	0.814	0.672	+17.4%
100	simple	3650 DAYS	5	5	1.461	1.143	+21.8%

100	window	1 DAY	20	5	0.697	0.545	+21.8%
100	window	30 DAYS	1	0	2.533	2.495	+1.5%

Table 6: Benchmark results for Refresh test cases, with specified dimension configurations.

CASES

SCENARIO	PARQUET_COUNT	OLD TIME (s)	NEW TIME (s)	% SPEEDUP
concurrent_feature_refresh	1	0.062	0.068	-11.2%
concurrent_feature_refresh	10	0.506	0.453	+10.3%
concurrent_feature_refresh	100	3.115	2.914	+6.4%
gc_pressure_refresh	1	0.142	0.137	+3.7%
gc_pressure_refresh	10	1.141	0.797	+30.2%
gc_pressure_refresh	100	6.183	4.966	+19.7%
high_retain_gc_pressure	1	0.153	0.155	-1.7%
high_retain_gc_pressure	10	1.292	0.859	+33.5%
high_retain_gc_pressure	100	8.623	6.215	+27.9%
multi_version_refresh	1	0.150	0.131	+12.7%
multi_version_refresh	10	1.055	0.513	+51.4%
multi_version_refresh	100	3.889	2.913	+25.1%
refresh_serve_interleaved	1	0.110	0.121	-10.1%
refresh_serve_interleaved	10	0.783	0.564	+27.9%

refresh_serve_interleaved	100	4.698	3.365	+28.4%
refresh_then_serve	1	0.037	0.037	+0.7%
refresh_then_serve	10	0.211	0.159	+24.7%
refresh_then_serve	100	1.231	0.937	+23.8%

Table 6: Benchmark results for Cases test cases, with specified dimension configurations.

Visualization

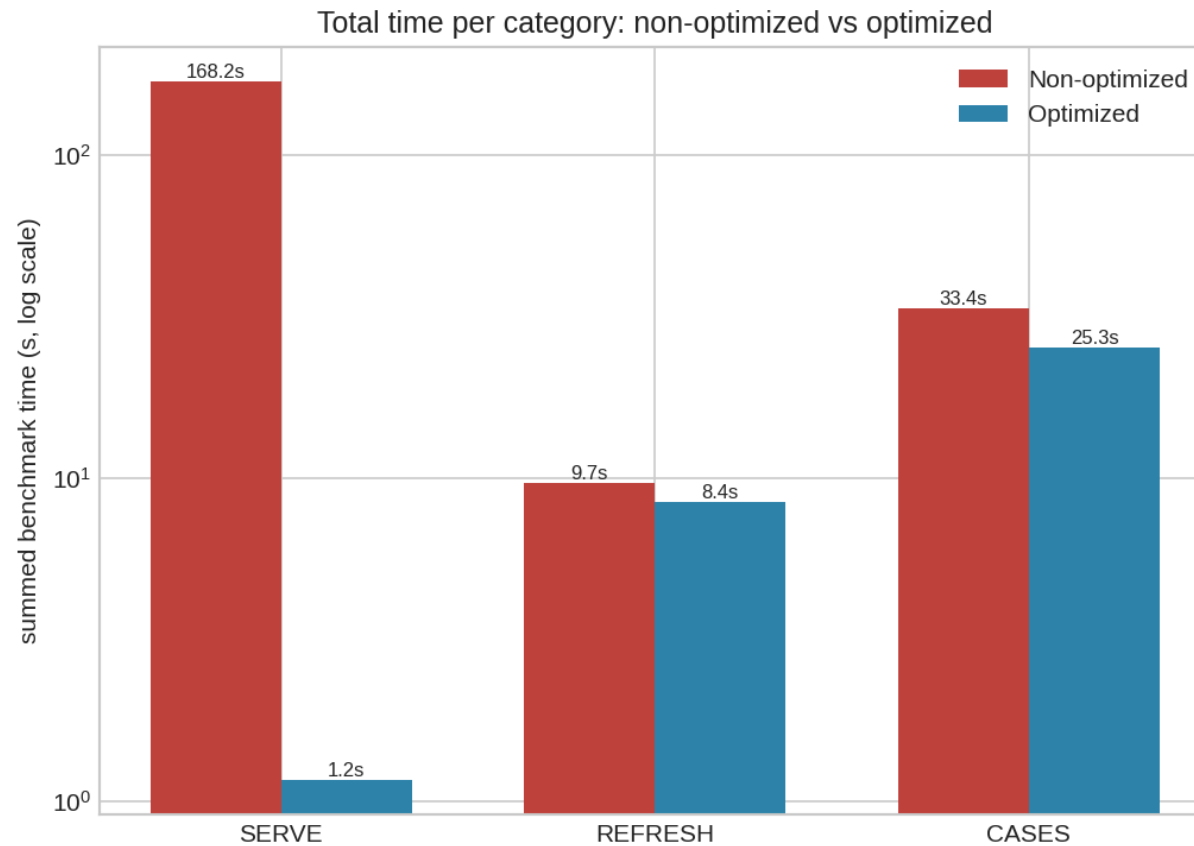


Figure 1: Bar chart comparison of accumulated run time of each benchmark category, with and without optimizations.

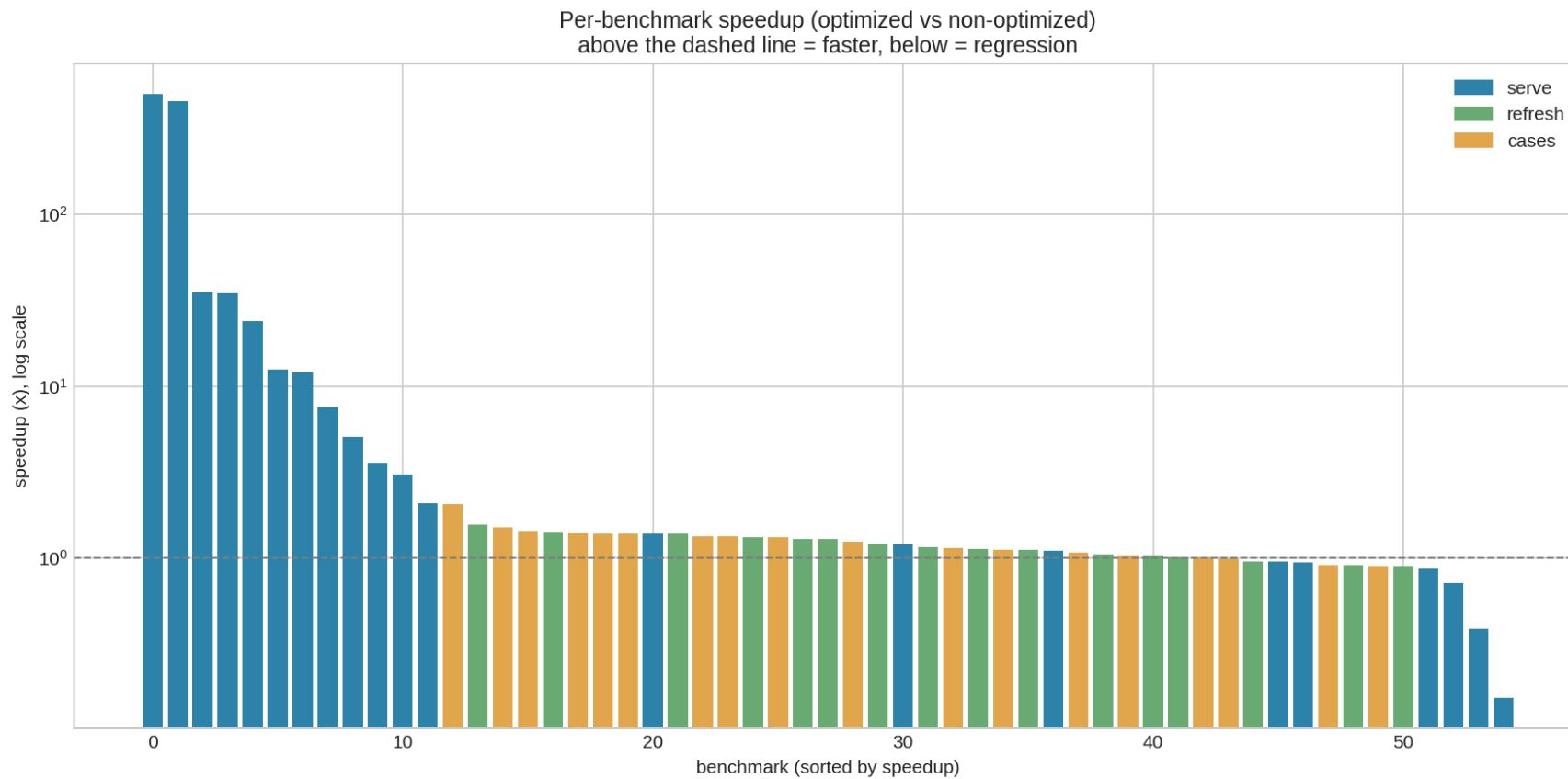


Figure 2: Bar chart comparison of per-benchmark speedup (by test case, coloured by category). Above 10^0 is a speedup.

CASES: time vs scale, non-optimized vs optimized (log-log)

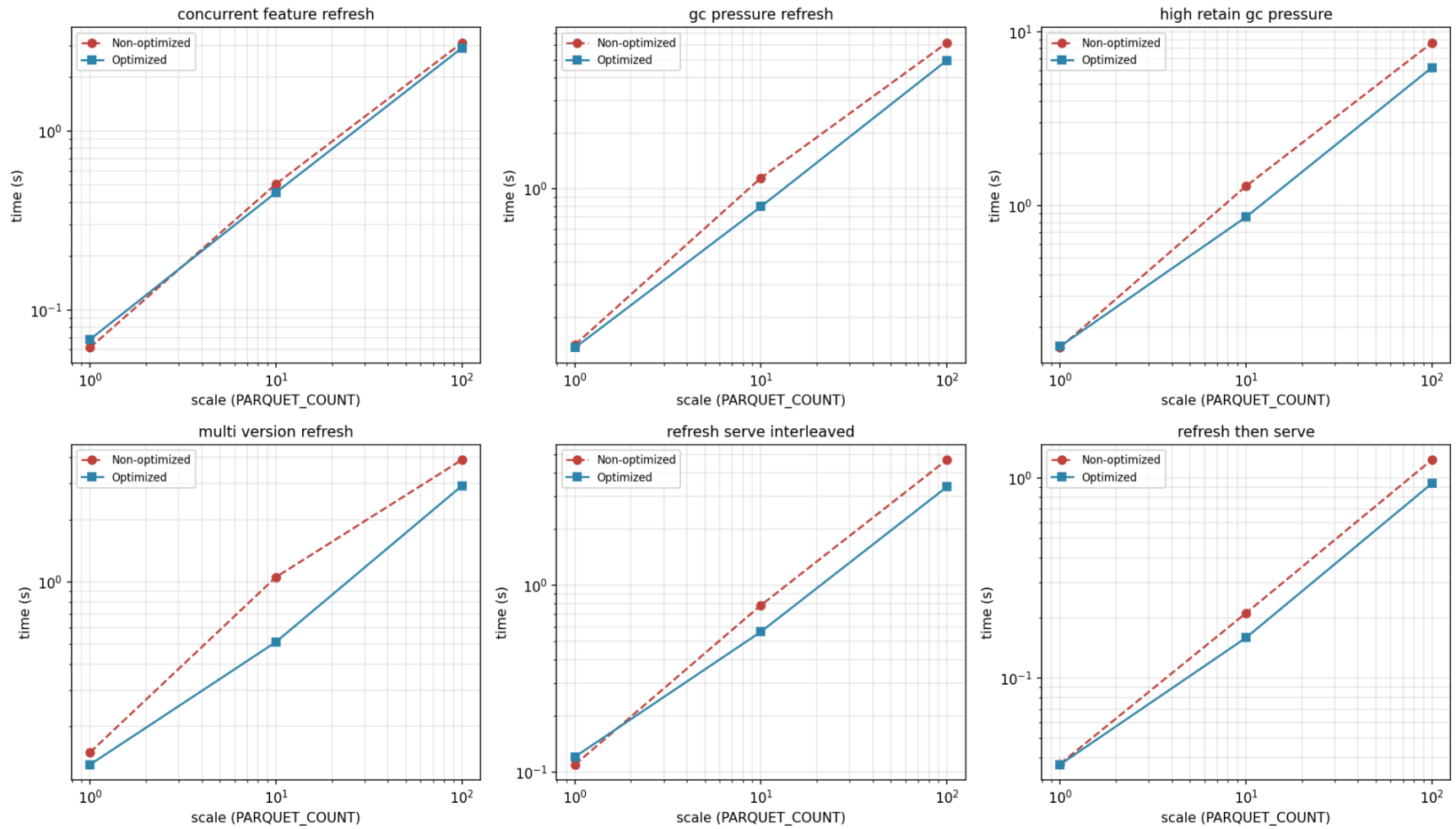


Figure 3: Line graph comparison of Cases test cases: time vs. scale (log-log)